ITIS4260 Project Machine Learning 1

April 9, 2025

1 Project Machine Learning 1

ITIS 4260 Efren Antonio

[2]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

[3]:
```
# Read in the data from the CSV file
df = pd.read_csv('datasets/wc1.csv')
pd.set_option('display.max_columns', None)
df.head()
```

[3]:

	inline_count	external_count	onclick_count	onload_count	onchange_count	avg_inline_script_block	avg_external_script_block	avg_onclick_count	avg_onload_count	avg_onchange_count	avg_cyc_complexity
0	21.0	23.0	1	131.0	0.0	0.0	662.062500	30737	4	16	0.812500
1	13.0	30.0	2	4.0	1.0	0.0	55.777778	2951	12	18	0.555556
2	0.0	3.0	1	0.0	0.0	0.0	207.000000	387	2	1	1.000000
3	21.0	11.0	1	10.0	1.0	0.0	104.800000	1505	2	5	0.400000
4	10.0	5.0	1	0.0	0.0	0.0	473.000000	4121	2	3	1.000000

```
   library_code_count   type
0                  13  alexa
1                  10  alexa
2                   1  phish
3                   2  alexa
4                   3  alexa
```

[4]:
```
# Convert categorical feature into dummy variables with one-hot encoding
df = pd.get_dummies(df, columns=['type'])
df.sample(3)
```

[4]:
```
       inline_count  external_count  onclick_count  onload_count  \
2805            5.0             0.0              1           0.0
12937          20.0            28.0              2           0.0
13193           4.0             6.0              1           0.0

       onchange_count  avg_inline_script_block  avg_external_script_block  \
2805              0.0                      0.0                   0.000000
12937             3.0                      0.0                 620.846154
13193             0.0                      0.0                1342.250000

       avg_onclick_count  avg_onload_count  avg_onchange_count  \
2805                   0                 0                   0
12937              22510                12                  13
13193              16530                 2                   4

       avg_cyc_complexity  library_code_count  type_alexa  type_phish
2805             0.000000                   0       False        True
12937            0.846154                  11        True       False
13193            1.000000                   4        True       False
```

[5]:
```
# Split dataset up into train and test sets,random_state is the seed for random␣
 ↪number generator
X_train, X_test, y_train, y_test = train_test_split(
    df.drop('type_phish', axis=1), df['type_phish'],
    test_size=0.33, random_state=133)
```

[6]:
```
# Initialize and train classifier model
clf = LogisticRegression(max_iter=10000).fit(X_train, y_train)

# Make predictions on test set
y_pred = clf.predict(X_test)
y_score2 = clf.predict_proba(X_test)[:,1]
```

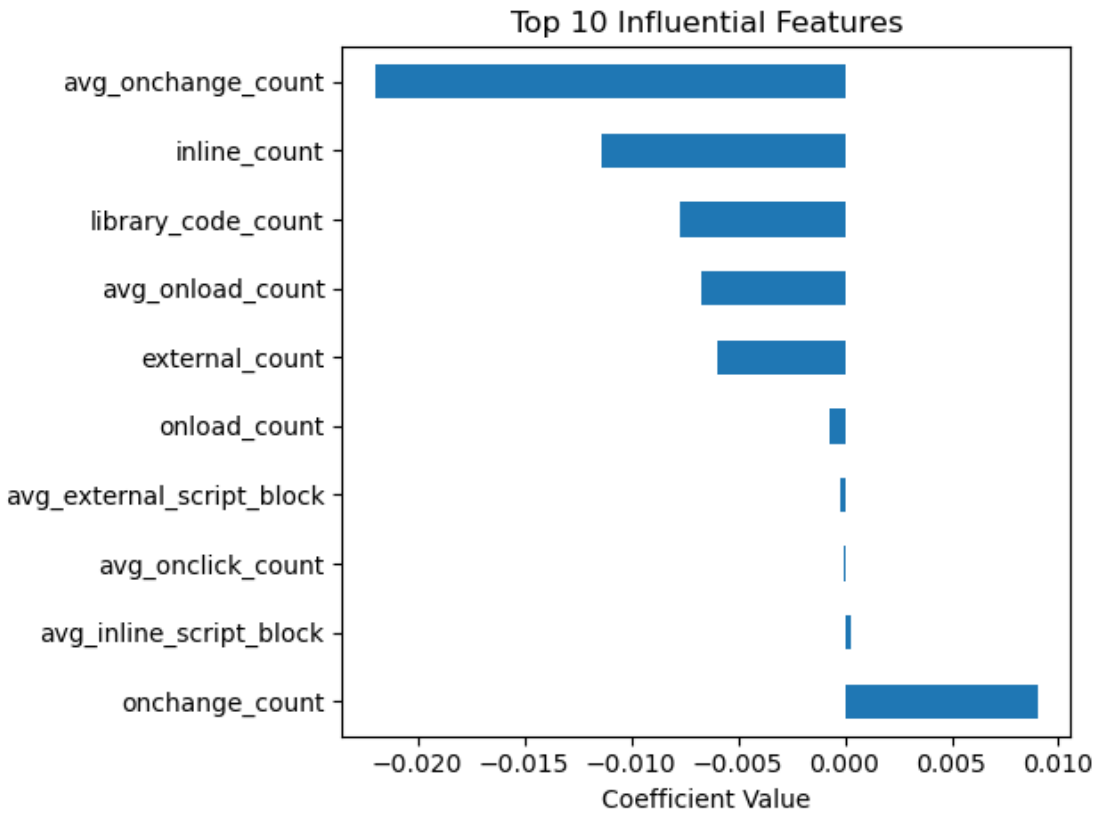
[7]:
```
# Compare test set predictions with ground truth labels
print(accuracy_score(y_pred, y_test))
print(confusion_matrix(y_test, y_pred))
```

```
1.0
[[5805    0]
 [   0 2192]]
```

[8]:
```
df.columns
print(clf.coef_[0])
```

```
[-1.14014504e-02 -5.96037148e-03 -4.70489595e-02 -6.73726361e-04
  9.00196967e-03  3.19618335e-04 -1.91025856e-04 -4.25291438e-06
 -6.69506986e-03 -2.19717179e-02 -5.24474406e-01 -7.72963046e-03
 -1.22598522e+01]
```

[9]:
```
X = ['inline_count', 'external_count', 'onclick_count', 'onload_count',
     'onchange_count', 'avg_inline_script_block', 'avg_external_script_block',
     'avg_onclick_count', 'avg_onload_count', 'avg_onchange_count',
     'avg_cyc_complexity', 'library_code_count', 'type']
feat_importances = pd.Series(clf.coef_[0], index=X)
feat_importances.nlargest(10).plot(kind='barh')
plt.title("Top 10 Influential Features")
plt.xlabel("Coefficient Value")
plt.tight_layout()
plt.show()
```



[10]:
```
from sklearn.model_selection import KFold

model_accuracy = []
kf = KFold(n_splits=10)
#split train data to train and validation
for train, val in kf.split(X_train, y_train):
    clf = LogisticRegression(max_iter=10000).fit(X_train.iloc[train], y_train.
 ↪iloc[train])
    y_pred = clf.predict(X_train.iloc[val])
    model_accuracy.append({'model': clf, 'acc': accuracy_score(y_pred, y_train.
 ↪iloc[val])})
    print(accuracy_score(y_pred, y_train.iloc[val]))

#choosing the highest accuracy model
clf_best = max(model_accuracy, key = lambda x: x['acc'])['model']

#run it on the test set
y_test_pred = clf_best.predict(X_test)
print('test accuracy:',accuracy_score(y_test, y_test_pred))
```

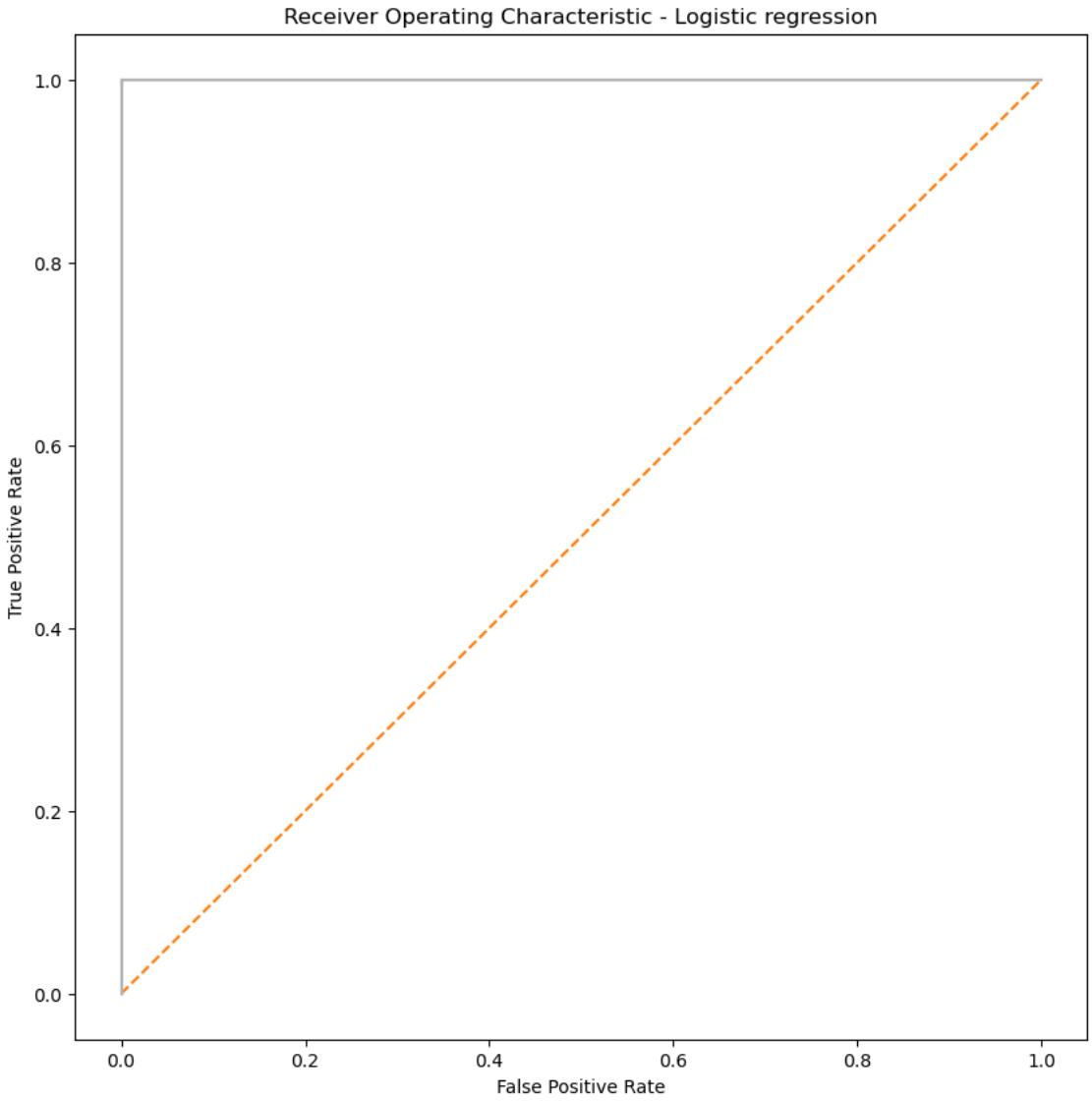
```
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
test accuracy: 1.0
```

[11]:
```
#feature selection
false_positive_rate2, true_positive_rate2, threshold2 = roc_curve(y_test,␣
 ↪y_score2)
print('roc_auc_score for Logistic Regression: ', roc_auc_score(y_test,␣
 ↪y_score2))
```

```
roc_auc_score for Logistic Regression:  1.0
```

[12]:
```
# Ploting ROC curves
plt.subplots(1, figsize=(10,10))
plt.title('Receiver Operating Characteristic - Logistic regression')
plt.plot(false_positive_rate2, true_positive_rate2)
plt.plot([0, 1], ls="--")
plt.plot([0, 0], [1, 0] , c=".7"), plt.plot([1, 1] , c=".7")
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
```

```
plt.show()
```



[13]:
```
cols_to_normalize = ['inline_count', 'external_count', 'onclick_count',␣
 ↪'onload_count',
                     'onchange_count', 'avg_inline_script_block',␣
 ↪'avg_external_script_block',
                     'avg_onclick_count', 'avg_onload_count',␣
 ↪'avg_onchange_count',
                     'avg_cyc_complexity', 'library_code_count']
df1=df
df1[cols_to_normalize]=(df1[cols_to_normalize]-df1[cols_to_normalize].min())/
 ↪(df1[cols_to_normalize].max()-df1[cols_to_normalize].min())
```

```
df1.sample(10)
```

[13]:

	inline_count	external_count	onclick_count	onload_count
16672	0.001570	0.000000	0.000000	0.000000
21474	0.006279	0.011385	0.000000	0.001981
18069	0.009419	0.005693	0.041667	0.000283
13365	0.000000	0.001898	0.041667	0.000141
22032	0.070644	0.113852	0.083333	0.015280
23190	0.012559	0.015180	0.041667	0.001132
19609	0.001570	0.000000	0.000000	0.000000
5101	0.004710	0.011385	0.041667	0.000000
15801	0.003140	0.001898	0.000000	0.000000
23656	0.003140	0.005693	0.041667	0.001698

	onchange_count	avg_inline_script_block	avg_external_script_block
16672	0.000000	0.000000	0.000000
21474	0.000000	0.000000	0.049588
18069	0.000000	0.000000	0.000803
13365	0.003165	0.000000	0.000000
22032	0.000000	0.035088	0.015077
23190	0.000000	0.035088	0.017460
19609	0.000000	0.000000	0.000000
5101	0.000000	0.000000	0.000000
15801	0.000000	0.000000	0.011183
23656	0.006329	0.000000	0.014612

	avg_onclick_count	avg_onload_count	avg_onchange_count
16672	0.000000	0.000000	0.000000
21474	0.048966	0.011765	0.028736
18069	0.000198	0.023529	0.005747
13365	0.000000	0.000000	0.000000
22032	0.173010	0.094118	0.224138
23190	0.019617	0.035294	0.028736
19609	0.000000	0.000000	0.000000
5101	0.000000	0.000000	0.000000
15801	0.001656	0.000000	0.005747
23656	0.006595	0.011765	0.011494

	avg_cyc_complexity	library_code_count	type_alexa	type_phish
16672	0.000000	0.000000	False	True
21474	0.600000	0.028571	True	False
18069	1.000000	0.009524	False	True
13365	0.000000	0.000000	False	True
22032	0.487179	0.180952	False	True
23190	0.000000	0.000000	False	True
19609	0.000000	0.000000	False	True
5101	0.000000	0.000000	False	True

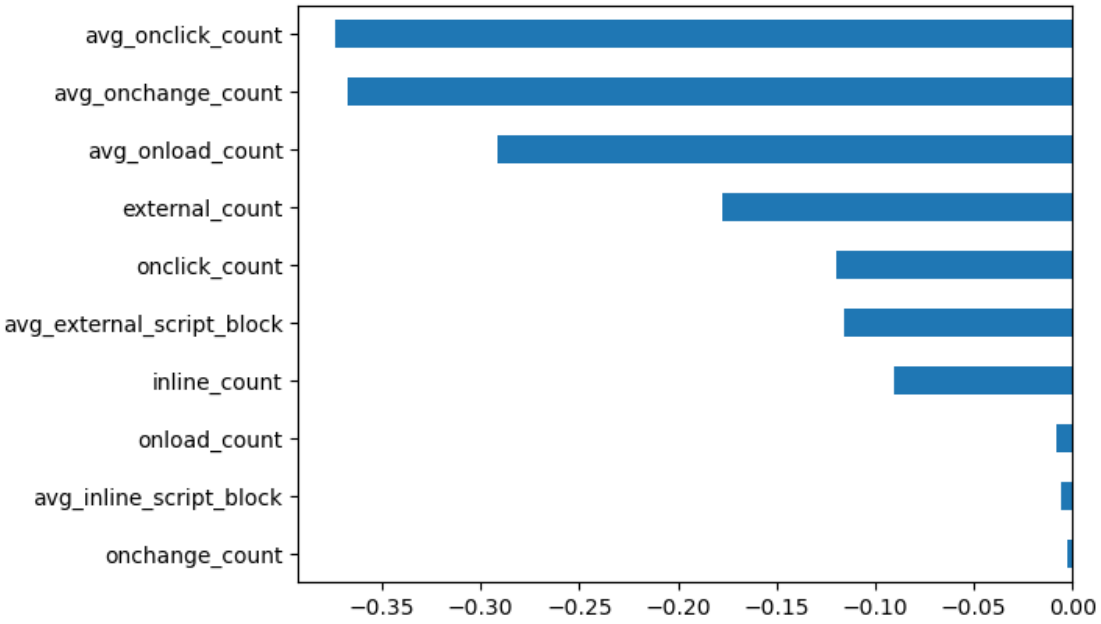
```
15801           1.000000            0.009524       False         True
23656           1.000000            0.019048        True        False
```

[14]:
```
# Split dataset up into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    df.drop('type_phish', axis=1), df['type_phish'],
    test_size=0.33, random_state=17)

# Initialize and train classifier model
clf = LogisticRegression(max_iter=10000).fit(X_train, y_train)

# Make predictions on test set
y_pred = clf.predict(X_test)
y_score2 = clf.predict_proba(X_test)[:,1]

# Compare test set predictions with ground truth labels
print(accuracy_score(y_pred, y_test))
print(confusion_matrix(y_test, y_pred))
```

```
1.0
[[5820    0]
 [   0 2177]]
```

[15]:
```
# Compute ROC curve and ROC area
false_positive_rate2, true_positive_rate2, threshold2 = roc_curve(y_test,␣
 ↪y_score2)
roc_auc = roc_auc_score(y_test, y_score2)

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(false_positive_rate2, true_positive_rate2, label=f'Logistic Regression␣
 ↪(AUC = {roc_auc:.2f})', linewidth=2)
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier (AUC = 0.5)')  #␣
 ↪Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('Receiver Operating Characteristic - Logistic Regression',␣
 ↪fontsize=14)
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()
```

[16]:
```
X = ['inline_count', 'external_count', 'onclick_count', 'onload_count',
     'onchange_count', 'avg_inline_script_block', 'avg_external_script_block',
     'avg_onclick_count', 'avg_onload_count', 'avg_onchange_count',
     'avg_cyc_complexity', 'library_code_count', 'type']
feat_importances = pd.Series(clf.coef_[0], index=X)
feat_importances.nlargest(10).plot(kind='barh')
plt.show()
```


avg_onclick_count
avg_onchange_count
avg_onload_count
external_count
onclick_count
avg_external_script_block
inline_count
onload_count
avg_inline_script_block
onchange_count
−0.35
−0.30
−0.25
−0.20
−0.15
−0.10
−0.05
0.00